Requirements Analysis and Specification Document (RASD)

Project: Air Quality Analysis using Dati Lombardia

Course: Software Engineering for Geoinformatics – Politecnico di Milano
Version: 0.1 – 27 May 2025

Revision History

Version	Date	Author	Notes

Table of Contents

1 Introduction 2

1.1 Purpose 2

1.2 Context and Motivations 2

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Scope and Limitations 3

2 Users 4

3 Use Cases 5

3.1 User Stories 5

3.2 Use Case Diagram 5

4 Functional Requirements – Modular Numbering 6

5 Non-Functional Requirements (NFRs) 8

6 Constraints and Dependencies 9

6.1 Technical Constraints 9

6.2 Data Constraints 9

6.3 Legal & Organisational Constraints 10

6.4 External Tool Dependencies 10

7 Future Considerations 10

8 Appendices and References 10

8.1 Open-Data Sources 10

8.2 Tools, Libraries & Frameworks 11

8.3 Internal Artefacts & Diagrams 11

8.4 Institutional & Academic Links 11

8.5 Acronyms & Abbreviations 12

1 Introduction

1.1 Purpose

This Requirements Analysis and Specification Document (RASD) defines the functional and non-functional requirements of the **"Air Quality Analysis using Dati Lombardia"** project, developed during the Software Engineering for Geoinformatics course at Politecnico di Milano (academic year 2024/25).

The goal is to deliver a client-server web application that lets users query, visualize and analyze air-quality sensor data published on the *Dati Lombardia* open-data portal.

The system will enable users to:

- **Query data** by time interval, administrative unit (municipality, province, region) and pollutant, with optional sorting and aggregation.
- **Visualise results** through interactive maps, charts and tables.
- **Perform basic analysis** such as trend detection, risk-level computation and indicator derivation.

1.2 Context and Motivations

Lombardy consistently reports some of the highest air-pollution levels in Italy, driven by traffic, industrial activity and high population density. Although the regional monitoring network publishes detailed measurements via *Dati Lombardia*, the raw datasets are difficult to explore for non-specialists.

The proposed application addresses this gap by providing an accessible interface aimed at citizens, researchers and public authorities. Specifically, the project seeks to:

- Promote environmental awareness through transparent, up to date information.
- Support evidence-based decision making in public health and urban-planning domains.
- Offer a modular architecture that can be extended to additional environmental datasets in future iterations.

1.3 Definitions, Acronyms, and Abbreviations

TERM / ACRONYM	DEFINITION
RASD	Requirements Analysis and Specification Document
API	Application Programming Interface – a set of endpoints through which the client dashboard interacts with the backend server
AQI	Air Quality Index – an indicator of overall air pollution levels

Dati Lombardia	The official open data portal of the Lombardy Region that provides access to public datasets including air quality measurements
PM10 / PM2.5	Particulate Matter with diameter less than 10 µm or 2.5 µm, major air pollutants
NO_2	Nitrogen Dioxide – a common pollutant measured by air quality stations
O_3	Ozone – a pollutant measured in tropospheric air quality analysis
Flask	A Python-based micro web framework used to implement the backend server
PostgreSQL/PostGIS	Relational database management system with spatial extensions for storing georeferenced data
JSON	JavaScript Object Notation – lightweight data-interchange format used in API responses
Jupyter Notebook	A web-based interface used for creating interactive dashboards with Python
UI	User Interface
UX	User Experience
Municipality / Province	Administrative units in Lombardy used for spatial analysis
Sensor	A device that measures air pollutants and transmits values to the central data system

Table 1. Definitions, acronyms and abbreviations.

Sensor: A hardware device located at monitoring stations, measuring air pollutant levels (e.g., PM10, NO_2) at regular intervals.
Algorithm: Backend logic that processes raw data, performs aggregation, and generates visualizations.
User Interface: The front-end dashboard component where users interact with the system (e.g., selecting filters, downloading data).

1.4 Scope and Limitations

The scope of this application is to provide an interactive dashboard that enables users to access, query, visualize, and download air quality sensor data collected from the Dati Lombardia open data portal.

In Scope

- Air quality measurements (e.g., PM10, PM2.5, NO_2, O_3) collected from fixed monitoring stations.
- Map and table views for any town or province in Lombardy.
- Fast filters for time, place and pollutant, plus basic stats (average, max, min).
- One-click export to CSV or PNG.
- Personal log-ins so you can save favourite queries.

- Manual import of historical files, with a first-cut auto-update script.

Out of Scope

- Weather data (e.g., temperature, wind speed) is not included in this version of the project.
- Real-time map-based forecasting or machine-learning prediction models are not implemented.

Limitations

- Dataset is limited to available records from Dati Lombardia; missing or delayed records might affect analysis.
- The application is optimized for desktop environments and is not designed for mobile devices.
- Current scope is restricted to the Lombardy region only; extension to other regions is possible but not planned in this project.
- Data integration from multiple sources and automation of continuous ingestion is optional (bonus feature).

---

2 Users

2.1 Users (User Classes)

A **user** is *any external role that exchanges information with the system*; a single real-world **user** (e.g. an ARPA analyst) may play one or more of those roles.
The system recognizes **two** human users, corresponding to the authentication states encoded in the ER-diagram's *Users* table.

User ID	Description	Typical real-world users	Main capabilities
Unauthenticated	Not logged in: either browsing anonymously or registered without an active session	Citizens, journalists, previously registered users	Preview maps and charts; access registration and login forms
Authenticated	Valid session after log-in.	Environmental agencies, health authorities, university staff/students	Full dashboard: advanced query, filter/sort/aggregate, download CSV/PNG, save personal views.

Table 2. Users and their functionalities.

---

3 Use Cases

3.1 User Stories

User stories provide informal, goal-oriented descriptions of software features from the perspective of end users, using the template:

As a [target user], I want to [action], so that [benefit].

ID	*Target user*	*Action*	*Benefit*
US-1	Unauthenticated	Register	Access full functionality
US-2	Unauthenticated	Explore general pollution data (map/table/chart)	Understand app value & decide whether to register
US-3	Unauthenticated	Log in	Access full functionality
US-4	Authenticated	Visualize data	Easier in-depth understanding
US-5	Authenticated	Query/filter/sort/aggregate data	Direct analysis, visualization, download
US-6	Authenticated	Download data	Side-processing of data
US-7	Authenticated	Analyse data	Derive indicators (overall air quality, risk, trends)

Table 3. Users and their user stories.

3.2 Use Case Diagram

The following use case diagram depicts a high-level overview of the relationship between use cases, users, and the application main features.

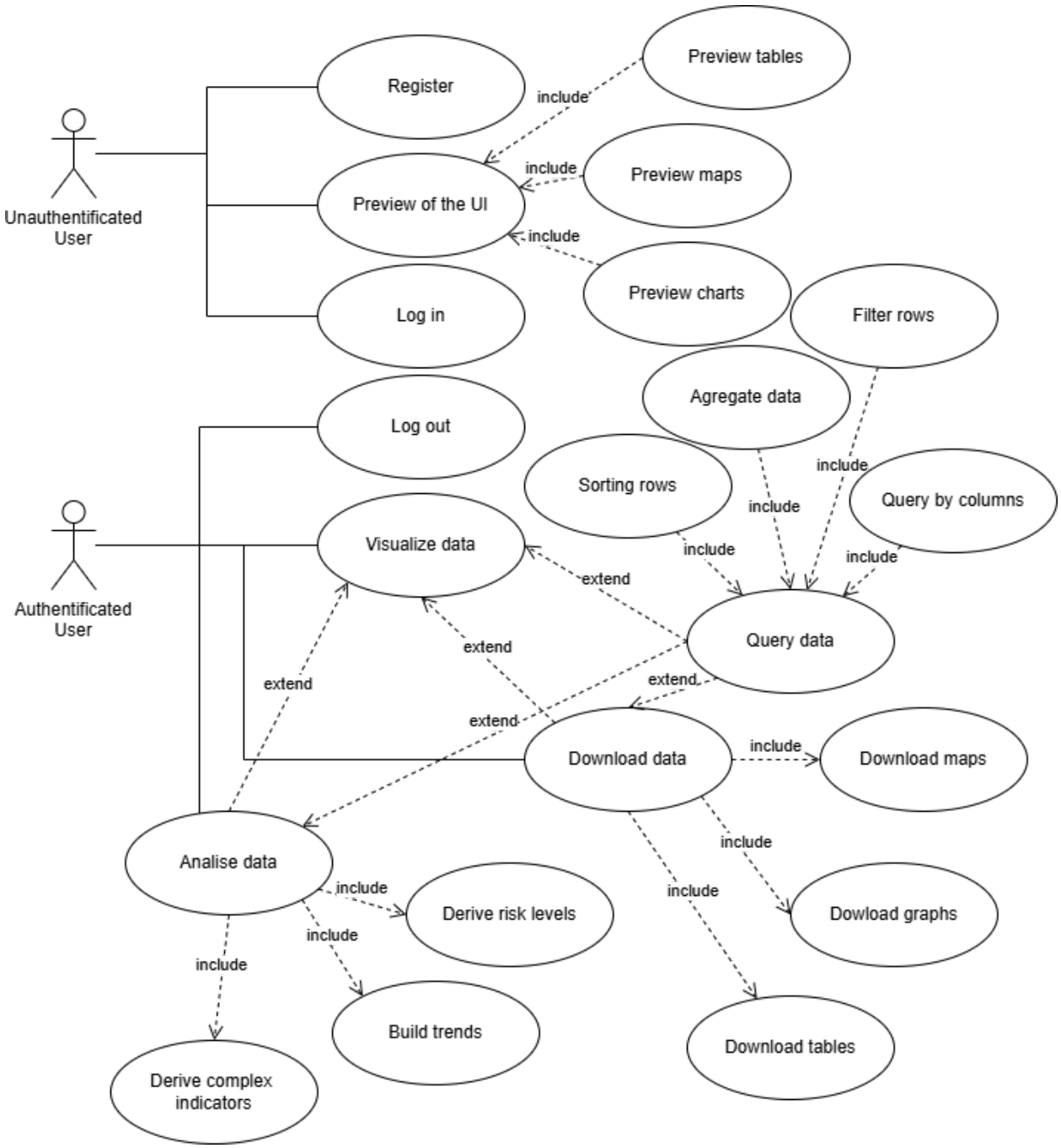


Figure 1. Use case diagram of the application

---

4 Functional Requirements – Modular Numbering

Functional requirements describe the specific behaviors and capabilities the system must support. Below, the requirements are grouped into modules and numbered for traceability.

F1 – User Authentication and Account Management

- **F1.1 User Registration** – The system shall allow unregistered users to create a new account by providing a unique username and a secure password.
- **F1.2 User Login** – The system shall authenticate registered users using valid credentials and provide access to personalized features.
- **F1.3 Reset Password** – The system shall enable users to reset their password via email verification if they forget it.
- **F1.4 Logout** – The system shall allow logged-in users to safely log out and invalidate their active session.

F2 – Data Access and Querying

- **F2.1 Sensor Selection** – Users shall be able to select one or more air-quality monitoring stations from a map or list interface.
- **F2.2 Time Range Selection** – The system shall allow users to filter data based on custom date and time ranges.
- **F2.3 Pollutant Type Filter** – Users shall be able to select specific pollutant types (e.g., PM10, NO_2, O_3) for analysis.
- **F2.4 Execute Query** – Upon selecting parameters, the system shall retrieve and display the requested data from the database.

F3 – Data Visualization and Export

- **F3.1 Table View** – The system shall present queried data in a tabular format.
- **F3.2 Chart View** – Users shall be able to view data trends in line or bar-chart format.
- **F3.3 Map View** – The system shall display data on an interactive geographic map with zoom, pan, and layer options.
- **F3.4 Export to CSV** – Users shall be able to export table data in CSV format.
- **F3.5 Export to PNG** – Users shall be able to export chart or map visualizations as PNG images.

F4 – Personalization Features

- ***F4.1 Save Map View*** *– Users shall be able to save the current map extent, active layers, and selected region under a custom name.*
- ***F4.2 Save Filter Configurations*** *– Users shall be able to store a set of filter parameters (pollutants, date range, sensors) for later reuse.*
- ***F4.3 Manage Favourite Sensors*** *– Users shall be able to add or remove sensors from a personal favourites list to speed up future queries.*
- ***F4.4 Auto-Load Saved Preferences*** *– When a user logs in, the system shall automatically restore the most-recently saved filters and map view.*

F5 – Notifications and Feedback

- ***F5.1 Login and Action Alerts*** *– Users shall receive confirmation messages after successful login, exports, and preference saves.*

- ***F5.2 Error Notifications*** *– The system shall inform users when actions fail (e.g., invalid query, empty result set, network timeout).*

---

5 Non-Functional Requirements (NFRs)

Every quality attribute is captured as a self-contained, testable requirement, using the same five-column template—ID │ Title │ Description │ Priority │ Acceptance Criteria—that the benchmark RASD employs.

ID	Category	Title	Priority	Description	Acceptance Criteria
NF-01	Interoperability	Data Retrieval & Integration	Medium	The system shall import hourly sensor measurements and metadata from the *Dati Lombardia* portal (CSV / JSON / API) without manual re-formatting.	A data-ingestion job runs and inserts ≥ 95 % of the published rows into the database with the correct schema.
NF-02	Maintainability	Modular Architecture	**High**	Code shall be organised in self-contained layers (data-access, services, presentation) so that new pollutants or widgets can be added without touching existing modules.	A new pollutant chart can be plugged-in by implementing one interface and all unit tests continue to pass.
NF-03	Security	Secure Data Storage	**High**	User credentials and configuration secrets must be hashed/salted; database volumes must support encryption-at-rest.	Pen-test confirms that raw passwords are never stored; connection strings are not exposed in logs.
NF-04	Security	GDPR Compliance	**High**	The platform shall process any personal data (e-mail, IP address) in accordance with EU GDPR.	Privacy notice is displayed; users can request deletion; audit shows no

					excessive data stored.
NF-05	Security	Password Policy	**High**	The system shall enforce a strong password: ≥ 10 chars, mix of cases/digits/symbols, renewed every 6 months.	Registration rejects *weakpass*; OWASP strength checks succeed.
NF-06	Compatibility	Cross-Browser Accessibility	Medium	UI must behave consistently on the latest versions of Chrome, Firefox, and Edge.	Selenium regression tests pass on all three browsers without layout shifts.
NF-07	Scalability	Infrastructure Support	Medium	Deployment artefacts (Docker Compose / Kubernetes YAML) shall allow horizontal scaling of the API and database.	Load test with 100 concurrent users keeps 95-percentile response < 3 s and CPU utilisation < 70 %.

---

6 Constraints and Dependencies

6.1 Technical Constraints

- **TC-01 Python + Flask only** – the backend must remain in Python 3.x with the Flask micro-framework.
- **TC-02 Local-first deployment** – evaluation runs on a single Windows host; cloud deployment is optional but not mandatory.

6.2 Data Constraints

- **DC-01 External dataset dependency** – application relies on the availability and stable schema of *Dati Lombardia* datasets.
- **DC-02 Data-quality gaps** – missing or delayed sensor records require preprocessing and interpolation.

6.3 Legal & Organisational Constraints

- **LC-01 Open-data licence** – only datasets published under CC-BY (or equivalent) may be used.

- **LC-02 Academic timeline** – features are frozen one week before final submission; no breaking changes afterwards.

6.4 External Tool Dependencies

Docker, PostgreSQL/PostGIS, GeoPandas, Plotly, and Folium must remain at the versions pinned in `requirements.txt`; breaking API changes will jeopardise the build pipeline.

---

7 Future Considerations

Mirroring the *Future Considerations* section in the sample document 毅filecite孿turn4file0傢, the following enhancements are earmarked for post-course development:

- **Advanced Analytics** – integrate machine-learning models to forecast pollution peaks and recommend mitigation strategies.
- **Real-time Streaming** – migrate from batch CSV ingestion to live MQTT/REST feeds for near-real-time dashboards.
- **Mobile-first UI** – responsive redesign or a dedicated mobile app to widen audience reach.
- **Collaborative Workspaces** – allow multiple agencies to annotate maps, share dashboards, and co-create reports.
- **External-System Integration** – expose a public JSON API so that third-party environmental portals can query aggregated indices.

---

8 Appendices and References

This section includes the external sources, references, and supporting materials used in the design, development, and documentation of the system. All links, datasets, diagrams, and tools listed here were either directly integrated into the system or influenced architectural and technical decisions. The section also includes internal deliverables, acronyms, and documentation standards that support transparency and traceability.

8.1 Open-Data Sources

ID	Dataset	Publisher	Why we use it
DS-01	*Air-Quality Measurements (hourly)*	Dati Lombardia	Core pollutant time-series (PM10, NO_2, O_3…) that feeds every map, chart, and CSV export.
DS-02	*Sensor Metadata*	Dati Lombardia	Station IDs, coordinates, install dates—lets us geolocate DS-01.

DS-03	*Administrative Boundaries*	Geoportale Lombardia	Municipality / province polygons for spatial joins and choropleths.

8.2 Tools, Libraries & Frameworks

Layer	What we picked	Job it does
Dev & Ops	Python 3.11, VS Code, Git + GitHub	Coding, linting, version control.
Backend / API	Flask	REST endpoints in a reproducible container.
Database	PostgreSQL 15 + PostGIS 3	Stores raw readings and runs spatial SQL.
Data wrangling	Pandas, GeoPandas, NumPy	Clean, aggregate, and merge sensor rows.
Visuals	Dash	Interactive charts, maps, and dashboard widgets.
CI/CD	GitHub Actions	Lint-and-test pipeline on every push.

8.3 Internal Artefacts & Diagrams

- System Architecture sketch – three-layer view (Client │ API │ DB).
- Simplified ERD – only core tables (*Sensor, Measurement, Pollutant, Municipality*) for clarity.
- Use-Case diagram – actors and log-in flow.

8.4 Institutional & Academic Links

- SE4GEO 2025 project brief – deliverable specs and deadlines (PoliMi).
- Scrum Handbook (WeBeep) – the lightweight agile recipe we follow.
- Politecnico academic calendar – milestones and exam dates.
- Private GitHub repo – source, issues, Kanban board.

8.5 Acronyms & Abbreviations

Term	Plain meaning
API	Application Programming Interface
CSV	Comma-Separated Values
CRUD	Create • Read • Update • Delete
DB	Database
GIS	Geographic Information System
IDE	Integrated Development Environment
JWT	JSON Web Token (auth)
MVP	Minimum Viable Product

Term	Plain meaning
REST	Representational State Transfer
UI / UX	User Interface / Experience
WSGI	Web Server Gateway Interface